

PostgreSQL & Python (Psycopg 3) Manual

Table of Contents

1. Installation & Setup
2. Connection Boilerplate
3. SQL Syntax Breakdown (The Queries)
4. Python Execution
5. CRITICAL: Security Measures

1. Installation & Setup

We are using the modern "Psycopg 3" library.

```
pip install "psycopg[binary]" flask
```

2. Connection Boilerplate

Include this function in your `app.py`. We use `dict_row` so you can access data by column name (e.g., `row['name']`) instead of index (`row[1]`).

```
import psycopg
from psycopg.rows import dict_row

def get_db_connection():
    # Update URI with: postgresql://[user]:[password]@[host]/[db_name]
    conn = psycopg.connect(
        "postgresql://postgres:password@localhost/school_db",
        row_factory=dict_row
    )
    return conn
```

3. SQL Syntax Breakdown

The strings inside `conn.execute("...")` are written in SQL, not Python. Here is how to structure them.

3.1 CREATE TABLE (One time setup)

Run this in your SQL tool (pgAdmin/DBeaver) before writing Python code.

```
CREATE TABLE students (  
    id SERIAL PRIMARY KEY,      -- Auto-incrementing ID  
    name VARCHAR(100) NOT NULL, -- Text, cannot be empty  
    grade INT                   -- Number  
);
```

3.2 INSERT (Adding Data)

To add a new row. The `%s` symbols are waiting for your Python variables.

```
"INSERT INTO students (name, grade) VALUES (%s, %s)"
```

INSERT INTO students: The target table.

(name, grade): The specific columns we are providing data for. (We skip 'id' because it is SERIAL/auto).

VALUES (%s, %s): Two placeholders for the two columns listed above.

3.3 SELECT (Reading Data)

To get data out.

A. Get Everything:

```
"SELECT * FROM students"
```

*****: Wildcard meaning "All Columns".

FROM students: The source table.

B. Filter (WHERE clause):

```
"SELECT * FROM students WHERE grade > %s"
```

WHERE: Acts as an `if` statement. Only rows matching the condition are returned.

grade > %s: Comparison logic.

C. Sorting (ORDER BY):

```
"SELECT * FROM students ORDER BY grade DESC"
```

ORDER BY grade: Sort by this column.

DESC: Descending (Highest to Lowest). **ASC** is Ascending (Lowest to Highest).

3.4 UPDATE (Editing Data)

Changing an existing row. **ALWAYS use a WHERE clause** or you will overwrite the entire table.

```
"UPDATE students SET grade = %s WHERE id = %s"
```

UPDATE students: Target table.

SET grade = %s: The column to change and the new value.

WHERE id = %s: Ensures we only update the specific student we want.

3.5 DELETE (Removing Data)

Deleting a row. **ALWAYS use a WHERE clause.**

```
"DELETE FROM students WHERE id = %s"
```

4. Python Execution

Combining the SQL strings with Python logic.

Example: The execution pattern

```
with get_db_connection() as conn:

    # 1. Provide the SQL string first
    # 2. Provide a Tuple (list) of variables second
    conn.execute(
        "INSERT INTO students (name, grade) VALUES (%s, %s)",
        ("Alice", 95)
    )

    # 3. IF writing data (Insert/Update/Delete), you MUST commit.
    conn.commit()
```

5. CRITICAL: Security Measures

⚠️ WARNING: SQL INJECTION

Never use Python f-strings or string concatenation (+) to put variables into a SQL command. This allows hackers to delete your database.

```
# ❌ WRONG (Dangerous)
conn.execute( f"INSERT INTO students VALUES ('{name}')" )

# ✅ CORRECT (Safe)
# Let the library handle the variables via %s
conn.execute( "INSERT INTO students VALUES (%s)" , (name,))
```